

CS 610 Semester 2024–2025-I: Assignment 5

Name: **Vinayak Devvrat**

Roll Number: **241110079**

November 17, 2024

Problem 1

The hash table has been implemented using a coarse-grained locking.

Parts (i) to (ii)

Please refer to the attached code.

Part (iii)

The throughput for `batch_insert()`, `batch_delete()` and `batch_lookup()` when using different hash functions is summarised below.

Parameters:

- Number of operations = 10000
- Number of threads = 16
- Number of runs = 2
- Percentage of add operations = 40
- Percentage of remove operations = 40
- Percentage of lookup operations = 20

Table 1: The throughput for various hash functions (s)

SNo	Hashing Technique	Operation		
		Insert	Delete	Lookup
1	Division Hash	0.2525	0.536	3.677
2	Multiplication Hash	0.0085	0.001	0.111
3	FNV Hash	0.01	0.001	0.112
4	Murmur Hash	0.007	0.0015	0.1105

Screenshots of the results of various hash functions have been attached.

Observation: Murmur hash gives the best performance and Division hash function gives the worst.

Part (iv)

Please refer to the attached code.

Part (v)

Please refer to the attached code.

Part (vi)

Based on the findings in Part (iii), I conclude that the **murmur hash function** gives the best performance in the Pthread Implementation. The following table gives a comparison between the PThread and the TBB implementations.

Parameters:

- Total number of operations = 10000000
- Number of INSERT operations = 6000000
- Number of LOOKUP operations = 2000000
- Number of DELETE operations = 2000000
- Number of threads = 16
- Number of runs = 1

Table 2: Comparative analysis of Coarse Grained PThread Implementation and Intel TBB Implementation (in s)

SNo	Operation	PThread	TBB
1	Batch Insert	43.055	0.042
2	Batch Delete	4.63	0.01
3	Batch Lookup	7.19	0.013

Screenshots have been attached as references.

Compilation Command: `g++ -std=c++17 -fopenmp -pthread -o prob1.out hash_table.cpp problem1.cpp -I/usr/local/tbb/include -L/usr/include/tbb -ltbb`

Compilation Command (For unit test cases): `g++ -std=c++17 -pthread -fopenmp hash_table.cpp unit_tests.cpp -o unit.out`

Run Command: `./prob1.out -ops=100000 -rns=2 -add=40 -rem=40 -tbb=0`

Set tbb to 1 to enable tbb implementation.

Run Command (For unit test cases): `./unit.out`

Problem 2

Parts (i) to (iv)

Please refer to the attached code for all the required implementations.

Part (v)

The results for several values of n ran against various number of threads are summarised below:

Table 3: Time taken to perform n operations (in ms)

SNo	n	Number of Threads				
		1	2	4	8	16
1	1e5	523	368	264	321	375
2	1e6	5145	3189	2678	3312	3760
3	1e7	59688	31352	28304	35010	40781

Observation: It has been observed that ideal number of threads for each of these configurations is 4. The time is least for this number in all the cases.

Compilation Command: `g++ -std=c++17 -pthread -fopenmp concurrent_stack.cpp problem2.cpp -o prob2.out`

Run Command: `./prob2.out -ops=1000000`

(Change the NUM_THREADS variable in the code for changing thread values.)